FalconTest

Automation Framework Guide

Meridian Quality Systems
Product Documentation · Version 4.2 · Beginner Guide

About This Document

Difficulty	★ Beginner
Best for	Parsing, Markdown, Tables, Basic Chunking, Basic Retrieval
Approximate Chunks	25
Estimated Embeddings	25
Recommended RAG Stages	Parsing → Chunking → Retrieval

Table of Contents

1. Introduction to FalconTest

FalconTest is a test automation framework developed by Meridian Quality Systems (MQS) for functional and regression testing of web and API-based applications. It is designed to give QA engineers a single framework that can drive browser-based tests, validate API responses, and orchestrate suite execution across distributed infrastructure.

FalconTest was first released as version 1.0 in 2019 and has since become the primary automation framework used across MQS's internal product teams, including the teams responsible for APIVerify and MockBridge.

> **NOTE:** FalconTest is a fictional product created for this documentation suite. Any resemblance to real automation tools is coincidental.

1.1 Design Goals

FalconTest was built around three design goals:

- **Simplicity** — a test should be readable by someone who did not write it.
- **Determinism** — a passing test should pass consistently, and a failing test should fail for a real reason.
- **Portability** — the same test suite should run locally, in a container, or on ClusterGrid without modification.

1.2 Where FalconTest Fits

FalconTest sits at the center of the MQS testing ecosystem. It integrates with APIVerify for contract-level API checks and with BuildFlow for continuous integration scheduling. Execution itself takes place on **ClusterGrid**, MQS's distributed execution infrastructure.

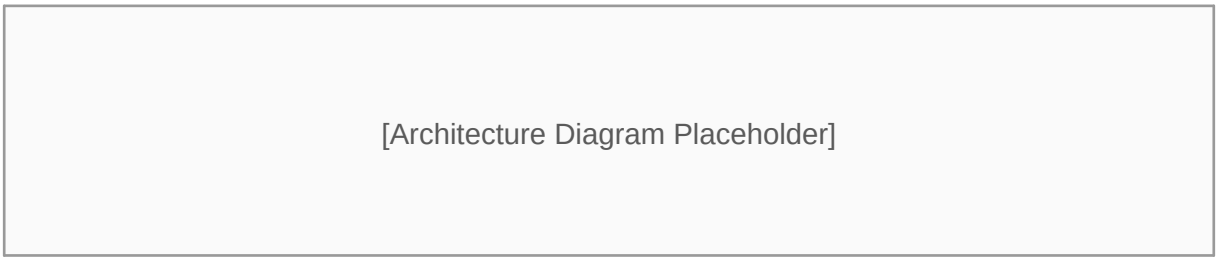

Figure 1: FalconTest's position within the MQS testing ecosystem.

2. Core Architecture

FalconTest is composed of four primary components.

Component	Responsibility
Test Runner	Loads and executes test definitions
Execution Cluster	Schedules and distributes test runs across available workers
Reporting Cluster	Aggregates results and generates reports
Config Loader	Resolves configuration from files, environment variables, and CLI flags

> **NOTE:** FalconTest documentation distinguishes between the **Execution Cluster**, which is responsible for running tests, and the **Reporting Cluster**, which is responsible for aggregating and publishing results. Both are part of ClusterGrid but serve different purposes. This distinction matters when troubleshooting — a slow test run is usually an Execution Cluster issue, while a missing report is usually a Reporting Cluster issue.

2.1 Test Runner

The Test Runner is the component that parses test files, resolves fixtures, and executes test steps in order. It supports both a declarative YAML test format and a scripted Python format.

2.2 Execution Cluster

The Execution Cluster receives test jobs from the Test Runner and distributes them across available worker nodes on ClusterGrid. It handles retries, timeouts, and worker health checks.

2.3 Reporting Cluster

The Reporting Cluster receives completed test results from the Execution Cluster and compiles them into the formats described in Section 7 (Reporting).

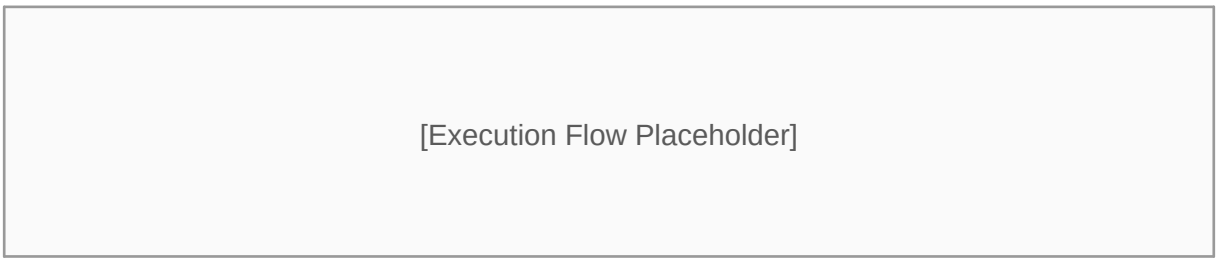

Figure 2: How a test job moves from the Test Runner through the Execution Cluster to the Reporting Cluster.

3. Installation

FalconTest is distributed as a Python package and can be installed using the standard package manager.

```
pip install falcontest==4.2.0
```

After installation, verify the CLI is available:

```
falcontest --version
```

Expected output:

```
FalconTest CLI 4.2.0
```

3.1 Minimum Requirements

Requirement	Minimum Version
Python	3.9
Operating System	Windows, macOS
Memory	4 GB RAM
Disk Space	500 MB

TIP: FalconTest 4.2 requires Python 3.9 or later. Attempting to install on Python 3.8 will fail with a dependency resolution error.

3.2 Initializing a Project

Once installed, a new FalconTest project is created with:

```
falcontest init my-project
```

This generates a default project structure:

```
my-project/
falcontest.yaml
tests/
example_test.yaml
fixtures/
reports/
```

4. Execution Modes

FalconTest supports two execution modes: **Passive** and **Adaptive**. Choosing the right mode affects both how quickly a suite runs and how it behaves under flaky conditions.

4.1 Passive Mode

In Passive mode, FalconTest executes each test exactly once, in the order defined, with no automatic retry and no adjustment to scheduling based on prior results. Passive mode is deterministic and predictable, which makes it well suited to smoke tests where a failure should be visible immediately rather than retried.

4.2 Adaptive Mode

In Adaptive mode, FalconTest monitors test outcomes as a suite runs and adjusts scheduling dynamically. If a test fails, Adaptive mode will consult the Smart Retry policy (see Section 5) before deciding whether to re-run it. Adaptive mode also reorders remaining tests to run known-flaky tests earlier in the suite, so that retries do not push the total suite duration past its allotted window.

4.3 Comparing Passive and Adaptive Modes

Aspect	Passive Mode	Adaptive Mode
Retry behavior	None	Governed by Smart Retry
Scheduling	Fixed order	Dynamically reordered
Best for	Smoke tests, CI gates	Full regression suites
Predictability	High	Moderate
Typical suite duration	Shorter, fixed	Variable

TIP: Use Passive mode for pull-request gating checks where a fast, deterministic signal matters more than tolerance for flakiness. Use Adaptive mode for nightly or scheduled regression suites where total pass rate matters more than individual run time.

5. Smart Retry

Smart Retry is the retry policy engine used by Adaptive mode. When a test fails, Smart Retry evaluates the failure against a small set of rules before deciding whether to retry it.

5.1 How Smart Retry Works

Smart Retry does not retry every failure. It first checks whether the failure matches a known **transient failure signature** — for example, a network timeout or a temporary element-not-found error in a browser test. If the failure matches a transient signature, Smart Retry schedules a retry. If the failure does not match a transient signature (for example, an assertion failure comparing expected and actual values), Smart Retry does not retry it, since retrying a genuine assertion failure would only waste execution time.

5.2 Retry Limit

In version 4.2, Smart Retry is configured with a **default retry limit of 3 attempts** per test. This value can be overridden per-suite in `falcontest.yaml`.

```
  smart_retry:
  enabled: true
  max_attempts: 3
  backoff_seconds: 5
```

WARNING: Setting `max_attempts` too high in a large suite can significantly increase total run time when many tests hit transient failures simultaneously, such as during a shared environment outage.

6. Configuration Reference

FalconTest configuration is defined in `falcontest.yaml` at the project root.

Key	Type	Default	Description
execution_mode	string	passive	Either passive or adaptive
smart_retry.enabled	boolean	false	Enables Smart Retry (Adaptive mode only)
smart_retry.max_attempts	integer	3	Maximum retry attempts per test
cluster.workers	integer	4	Number of ClusterGrid workers to request
report.format	string	html	Output format: html, json, or junit

Example configuration:

```
execution_mode: adaptive
smart_retry:
enabled: true
max_attempts: 3
cluster:
workers: 8
report:
format: html
```

7. Reporting

After a suite completes, the Reporting Cluster compiles results into a report. FalconTest supports three report formats.

Format	Use Case
HTML	Human-readable report with pass/fail breakdown and timing charts
JSON	Machine-readable output for downstream tooling
JUnit XML	Compatible with most CI dashboards, including BuildFlow

> **NOTE:** JUnit XML output from FalconTest is compatible with BuildFlow's built-in test result visualization.

8. Best Practices

- Keep Passive mode as the default for any pipeline stage that gates a merge.
- Reserve Adaptive mode with Smart Retry for scheduled, unattended runs.
- Set `cluster.workers` based on the number of independent test files, not the number of individual test cases.
- Review Smart Retry's transient-failure signatures periodically; a signature that was accurate at release time can go stale as the application under test changes.

9. Glossary

Term	Definition
ClusterGrid	MQS's distributed execution infrastructure that runs FalconTest jobs
Execution Cluster	The ClusterGrid component that schedules and runs tests
Reporting Cluster	The ClusterGrid component that aggregates results into reports
Smart Retry	FalconTest's retry policy engine, used in Adaptive mode
Transient failure signature	A pattern Smart Retry uses to identify failures worth retrying
Passive Mode	Execution mode with no retries and fixed test ordering
Adaptive Mode	Execution mode with dynamic scheduling and Smart Retry support

10. Questions to Try

The following questions are provided to help you explore this document using your own RAG pipeline. No answers are provided here — try retrieving and generating answers yourself.

1 What is FalconTest Smart Retry?

2 How does Adaptive mode decide whether to retry a failed test?

3 Compare Passive and Adaptive execution modes.

4 Which execution mode is recommended for scheduled regression suites?

5 What is the default value of `smart_retry.max_attempts` in version 4.2?

6 Which Cluster is responsible for test execution?

7 Does FalconTest support Linux?

8 What are the minimum system requirements to install FalconTest?

9 What report formats does FalconTest support?

10 What is the difference between the Execution Cluster and the Reporting Cluster?